C04｜几步修改怎样一起成功或撤回

一次确认需要共同留下：**预约存在、时段 booked、一条确认事件**。三条语句各自合法，不能证明这三处已经一起完成。

语句执行、错误拒绝与整组提交，差在哪？

当前问题	关系或短用法	记到什么程度
新增、修改和删除分别用什么？	INSERT 新增行；`UPDATE ... SET ... WHERE ...` 修改符合条件的行；DELETE 移除行。	逐渐认熟动作与目标条件；更新或删除不能漏掉本来需要的 WHERE。
怎样把几步归为一件事？	BEGIN → 预约 / 时段 / 事件 → COMMIT；失败走 ROLLBACK，撤回本事务的相关改变。	**业务完成条件决定事务边界。** 一条语句执行过，不等于整组提交了。
事务里已经读到 A1，就是长期保存了吗？	当前连接能读到自己的未提交写入，用于下一步写事件；成功提交才完成这组改变。	分清当前事务内可见与已提交。其他连接已有的旧读取事务，还涉及隔离，不由这一个小例证明。
第三步 CHECK 失败，为何还需回滚？	本例默认的约束失败通常只拒绝当前语句，前两步可能仍留在未结束事务中；捕获后照常 COMMIT 会留下半成品。	**约束发现坏状态，事务撤回相关改变。** 出错和整体撤回分别找证据。
三次 execute 没报错，函数就能返回成功吗？	COMMIT 本身也可能失败，所以应放在失败处理范围内；必要时回滚仍在进行的事务，再交回异常。	把“提交成功”作为正常出口；整段 try / rollback / raise 配方查原章。

本章独立程序设置 Python 3.12+ 的 `autocommit=True`，使用 `execute('BEGIN')`、`execute('COMMIT')`、`execute('ROLLBACK')` 显式管理事务。在这个模式下，Python 的 `connection.commit()`、`connection.rollback()` 不起作用。**连接模式与提交方式必须配套**；C05 的应用连接设置与 C06 的独立实验不同，不能交叉拼用。

事件表的整数 `id` 标识一条事件；预约 ID 标识预约；request ID 关联一次处理，一批可写多个共用它的事件。这里事件没有操作者字段，不能凭这些列回答“哪个人确认”。

回看：「一次“确认成功”究竟要求哪几处同时改变？」「为什么三条合法的 SQL 还不够？」「事件写坏时，预约为什么不会残留？」「怎样把这条关系写成能够运行的程序？」。

写前看见可用，写入时还要确认什么？

当前问题	关系或短用法	记到什么程度
预览过 SL1 是 available，UPDATE 还需条件吗？	真正写入仍用 `WHERE id=? AND status='available'`，因为两次操作之间状态可能变化。	预览帮助认目标，**写入条件保护当前动作**；不把预览结果当永久有效。
UPDATE 没报错，却没占到时段，怎么看？	检查返回游标的 `rowcount`；本例要求恰好 1 行，否则抛错，让外层撤回先插入的预约。	认熟“语句执行成功”与“命中所需行”不同；零行不是自动抛出的异常。
第一条写入就失败，是否还会更新时段？	不存在的时段可被外键先拒绝；已被其他预约使用可被 UNIQUE 先拒绝；存在且未占用但非 available，才可能到 UPDATE 命中零行。	沿**最早拒绝的位置**看后续有没有执行，不把所有失败都叫更新失败。

本章绑定参数的短片段如下，依赖原章已经建立的 `connection`：

> Python 示例
>
> ```
> changed = connection.execute(
>     "UPDATE slots SET status='booked' WHERE id=? AND status='available'",
>     (slot_id,),
> )
> ```

`?` 从后面的元组接收值，单元素元组需要逗号。这个片段只执行一次条件更新；完整确认还要把它与预约、事件放进同一事务，并检查 `changed.rowcount`。

完整 `setup`、`write_confirmation`、`confirm` 与状态读回查原章。它们验证指定的单连接成功和失败，没有证明所有并发交错，也不保证同一动作重放自动只有一次效果。

回看：「怎样把这条关系写成能够运行的程序？」。

批量导入、取消与自动规则，各自保护什么？

当前问题	关系或短用法	记到什么程度
CSV 里出现 thirty，应该先塞进正式时长吗？	暂存表保留原始文字和行号；准备阶段解析分钟与日期，核对工单客户、权威时段内容、可用状态和批内重复。	**接收原文不等于接受业务记录。** 先保留能定位的来源，再决定能否写入。
第二行坏了，第一行能留下吗？	本例要求整批成功：准备阶段有错就不交出可写结果；正式写入中出错也回滚整批。	完成口径决定边界，不能在每成功一行后提交。若改成逐行接收，需要另一套结果约定。
CSV 重复了时段日期和时长，能顺手覆盖 slots 吗？	正式 slots 已保存权威值；CSV 字段在本例用于核对它指向的时段，不能改写权威事实。	分清来源要验证什么；事前检查通过，也不能替代正式写入时的条件与约束。
取消预约为什么还保留原行？	confirmed 改 cancelled，并同事务写取消事件；有效看板需明确过滤 cancelled。	**状态改变不等于从未发生。** 本例 SL1 仍 booked，历史 A1 仍占 slot_id 的唯一范围，不承诺重新放号。
取消后为什么仍删不了客户？	工单直接引用客户，预约再引用工单；RESTRICT 保护这条历史链。CASCADE 会随父行删除关联子行。	看真实引用和删除范围；取消不解除外键，不为让 DELETE 通过就改成级联。
CHECK 允许 confirmed，为何还需防止取消后恢复？	CHECK 限制新值；trigger 可用 `OLD.status`、`NEW.status` 判断状态转换，并在指定写入时自动运行。	需要跨入口成立且依赖前后值的规则时再查 trigger；`RAISE(ABORT)` 拒绝语句，不替代外层事务。

批量的暂存 SQL 与解析、写入程序相邻查用。原章暂存行在业务事务开始前写入 TEMP 表，导入回滚后仍留在当前连接；关闭连接后消失。它不是自动持久保存的待办队列。

取消后若要重新预订，同步考虑历史保留、时段状态与唯一规则。只把 booked 改 available 会让展示与数据库约束矛盾。trigger 的完整定义也按需查；确认的三步业务写入仍由 Python 明确执行，不会因有 trigger 自动补齐。

回看：「三行导入，第二行写着 thirty，第一行能先留下吗？」「取消预约为什么不直接 DELETE？」「一条规则必须对每个写入入口成立，何时需要 trigger？」。